STANCE: Neural Network Report

ME5418 Group 42 · Vishal Mangla, Mukul Yadav, Muthaiah Mahadevan

1 Overview

We have added two neural-network policies for the STANCE environment of Report 1, and we have trained both with PPO. **Network A** is a feed-forward MLP over the last 10 sensor frames, while **Network B** is a recurrent LSTM that builds its own memory of the stance. Both set the ankle's target angle, stiffness and damping (θ_d, K, B) as a small *residual* around a fixed starting action. **Result:** in our runs the LSTM scored a higher return than the best fixed action on unseen episodes in all three seeds (+1.9 to +3.9) at slightly lower survival. The MLP was on par, and neither network did better on the stress test ground.

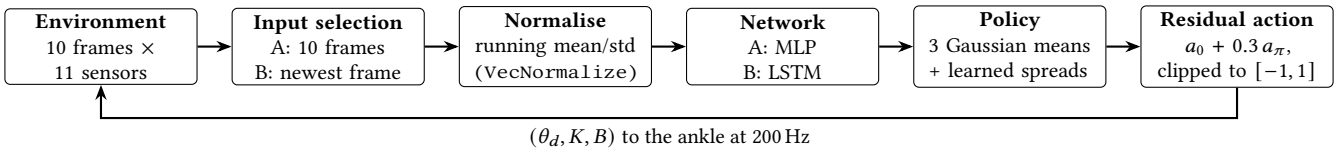
2 Code explanation

2.1 Organisation

Every command runs from `Network_Agent/`. `agent/train.py` is the single training entry point. It loads a network module that provides a fixed set of names (layers, inputs, algorithm, evaluation adapter), so both networks are trained and scored by exactly the same code.

Path	Responsibility	Author
`common/`	Shared and frozen: loads the environment and converts actions, selects input frames, the shared evaluator, policy adapters. The environment (`Gym/stance_env/`) is unchanged from Report 1.	All
`networkA_mlp/network.py`	Network A: history MLP and its input wrapper.	Muthaiah
`networkB_lstm/network.py`	Network B: frame encoder and recurrent actor–critic.	Mukul
`agent/`	Training settings (config v1), residual action, training and evaluation, learning curves.	Vishal
`demo_network.py`, `submission/`	Demo on the saved weights, plus the final weights and normalisation statistics.	Vishal

2.2 Data flow



We centre the policy on the anchor a_0, which is $(-14^\circ,\ 250.6\,\mathrm{N\,m/rad},\ 6.38\,\mathrm{N\,m\,s/rad})$, so an untrained network already behaves like a reasonable stance and learning starts from there. The largest nudge, 0.3 normalised units, is 15% of each range (K and B on a log scale). We also give the policy a critic head for the state value, used in training only.

2.3 The two networks

	Network A (MLP)	Network B (LSTM)
Input per step	10 frames × 11 = 110 values	newest frame only, 11 values (memory is the LSTM's)
Layers	separate actor and critic trunks 110 → 256 → 256 → 64 (tanh)	shared encoder 11 → 64 (tanh), then separate actor and critic LSTMs (64 units, reset each episode); 64 → 64 (tanh)
Parameters, time	221,575 (about 6.8 min per 300k-step run)	75,911 (about 17 min per run)

The ground's stiffness, yield point and dent history never appear in one frame. Network A sees a fixed 10-frame window, whereas Network B can accumulate evidence over the whole 100-step stance.

2.4 Training: forward and backward pass

In the *forward pass* the network gives the action distribution and the value, and in the *backward pass* we minimise the PPO loss

$$\mathcal{L} = -\,\mathbb{E}_t\left[\min\left(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right] + 0.5\,\mathbb{E}_t\left[\left(V(s_t) - \hat{R}_t\right)^2\right],$$

where ρ_t is the new-to-old policy probability ratio and $\epsilon = 0.2$ (entropy weight 0). For Network B, gradients flow back through time within each stored sequence (`RecurrentPPO`). We train both networks with one setup, **config v1**: 300k steps, 8 environments × 256 steps per update, batch 512, 10 epochs, Adam, learning rate 3×10^{-4}, $\gamma = 0.99$, GAE $\lambda = 0.95$, and gradient clip 0.5. In the demo (`demo_network.py`) we repeat one such step on a 128-step rollout without changing the saved weights. Every layer receives gradient, most of all the first feature layer and the action head (about 1.9 and 1.7 for the MLP). The total norm (3.2 for the MLP and 2.7 for the LSTM) is clipped to 0.5, which scales the update by about 0.15–0.19, and ten Adam steps on the batch lower the loss. For the LSTM the recurrent weights (`weight_hh`) also receive gradient.

2.5 Results

An episode is one simulated stance: a random ground is drawn and the policy sets the ankle at every step until the robot falls or 100 steps pass. All numbers have been computed with the shared evaluator (200 episodes per ground, seeds 1000–1199, deterministic policy) and averaged over three training seeds. Unseen episodes are new draws from the training ranges.

	Unseen episodes (training ranges)				Stress test ground (out of range)	
Controller	Survival	Toe peak (BW)	Return	vs fixed	Survival	Return (vs fixed)
Best fixed action ($-20°$, 500, 10)	92.5%	1.739	87.90	—	50.5%	78.57
Network A (MLP)	88.3%	1.720	89.01	+1.11	48.0%	78.26 (−0.31)
Network B (LSTM)	90.2%	1.699	90.87	**+2.97**	44.3%	77.11 (−1.46)

- **Unseen episodes (training ranges):** in our runs the LSTM scored higher than the best fixed action in all three seeds (+1.86 to +3.90 return), while the MLP was within noise. The LSTM's gain appears to come from a smaller force excess above the 1.58 BW level that the reward penalises, not from a lower toe peak (1.70 vs 1.74 BW).
- **Stress test ground (out of range):** this ground lies mostly outside the training ranges, and even the best fixed action chosen per episode in hindsight survives only 58.5% there. Neither network did better than the fixed action. We skip toe peak there, since the yield force caps it at 0.704 BW for every policy.
- **Memory ablation** (1 / 3 / 10 frames, 3 seeds): for Network B, more history had no clear effect on unseen episodes (survival 87.8 / 90.2 / 90.7%) but tended to help on the stress test ground (45.7 / 47.0 / 52.7%, return vs fixed −1.75 / −0.29 / +1.59). Network A did not (90.0 / 88.3 / 87.5% and 45.7 / 50.0 / 42.2%, return vs fixed −1.21 / +0.20 / −2.81), so the trend is weak.

3 Existing codes and libraries

Library	Used for	What we did to make it work
PyTorch 2.14.1	Layers, autograd, Adam.	We wrote both network bodies as feature extractors (`HistoryMLP`, `FrameEncoder`).
Stable-Baselines3 2.9.0	PPO, actor–critic policy, `VecNormalize`.	We plugged our bodies in through `policy_kwargs` and saved the normalisation statistics with the weights so the demo reproduces them.
sb3-contrib 2.9.0	`RecurrentPPO`, LSTM policy.	We carry the LSTM state between steps and reset it at each episode start, in training and in our evaluation adapter.
Gymnasium 1.4.0	Wrappers and spaces.	We implemented input selection and the residual action as wrappers, so the environment code is unchanged.

Additionally, MuJoCo 3.15.0 (physics, OpenAIGym Report), NumPy and Matplotlib (demo plots).

4 Reflections and lessons learned

4.1 Changes from the proposal

Item	Proposal	Current implementation
Architecture	Two-branch CNN + MLP, plain MLP as reference.	We use a plain MLP (A) and an LSTM (B), because the LSTM can remember the whole stance, not only a 10-frame window.
Action output	Gaussian means squashed to each range.	We use a residual nudge around a_0 (at most 15% of each range). Plain PPO varied by seed (88.5 / 57.5 / 17.0% survival), whereas the residual gave 86.5–92.0%.
Baseline	Fixed-gain FSM impedance controller.	We compare against the best of 45 fixed actions, and none of 36 two-phase schedules beat it.
Claim	RL improves on classical control.	Narrower: survival is near its ceiling (92.5 vs 93.5%), so we only claim a higher return at slightly lower survival. In our runs the LSTM did this in all three seeds (+3.0 return, +2.1 on 500 fresh episodes), whereas the MLP did not.
Evaluation	Held-out test ground.	Unseen episodes in the training ranges (main result) plus the out-of-range ground as a stress test. We added the split after seeing the results, and no number changed.
Metrics	Mean ± std, fall type, peak GRF vs stiffness.	We report means only, and fall type and force against stiffness will move to the agent stage.

4.2 Towards the final version (6 Nov)

- **Agent stage:** Our results support Network B (LSTM) as the primary network for the agent stage. We will add the final weights, a rollout video, the `render()` fix and a final benchmark (more seeds, remaining metrics).
- **Impact weight 2.94 → 6 (exploratory):** both networks beat the best constant action on unseen episodes in all three seeds (+3.6 to +9.3 return) but not clearly on the stress test ground.
- **Generalisation:** we plan a wider training ground, a fresh out-of-range set and more seeds, and will choose the history length on unseen episodes.

AI-use declaration. We used Claude (Anthropic) to draft and help debug the code, and to draft and generate clean tables and figures for this report. We are completely responsible for the final content and quality of the submitted work.